

# 《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

## 实验名称：

Shellcode 编写与编码

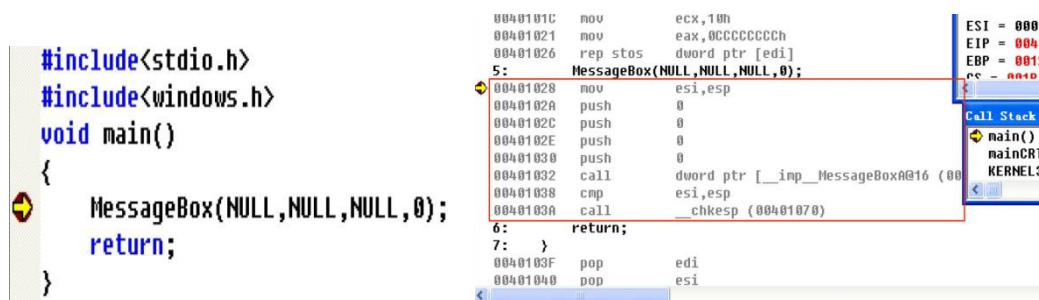
## 实验要求：

复现第五章实验三，并将产生的编码后的 shellcode 在示例 5-4 中进行验证，阐述 shellcode 编码的原理、shellcode 提取的思想。

## 实验过程：

### 1. 编写及提取 Shellcode 代码

汇编语言直接编写比较复杂，我们可以通过编写 C 的代码反汇编，观察并仿照得到的汇编代码结构就可以比较简便地编写汇编代码。例如，我们编写 5-2 中的代码（如左下图所示），进行反汇编，可以看到汇编代码的结构（如右下图所示）。

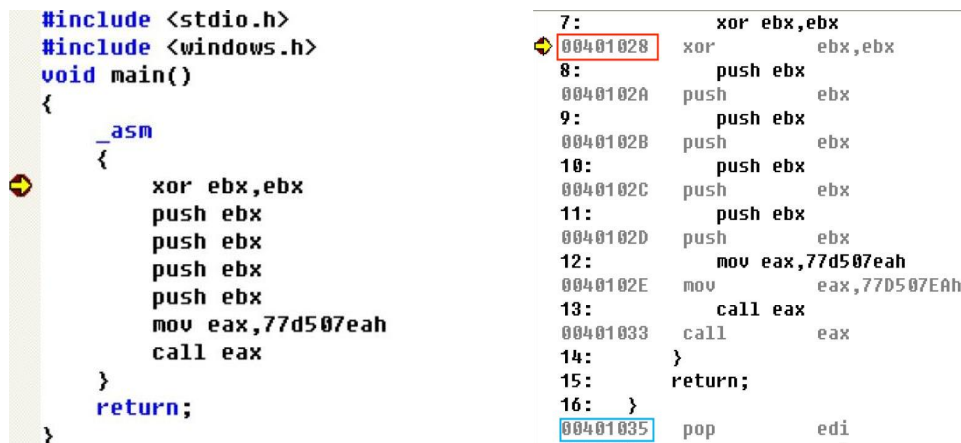


接下来我们尝试在 C 语言中插入汇编代码来复刻上述代码（MessageBox），不过在此之前，需要先得到 MessageBox 的地址，如左下图所示：



运行程序，如右上图所示，得到 MessageBox 的地址为 0x77d507ea。

下面就可以使用\_asm 语句在 C 语言中插入汇编代码，如左下图所示。这里汇编语言就是仿照上面我们观察到的汇编结构所写，其中，ebx 自身异或后 push ebx 就对应着 push 0（这是由于指令中不允许出现 0 字节）。然后编译，运行，成功出现弹窗，证明代码无误。



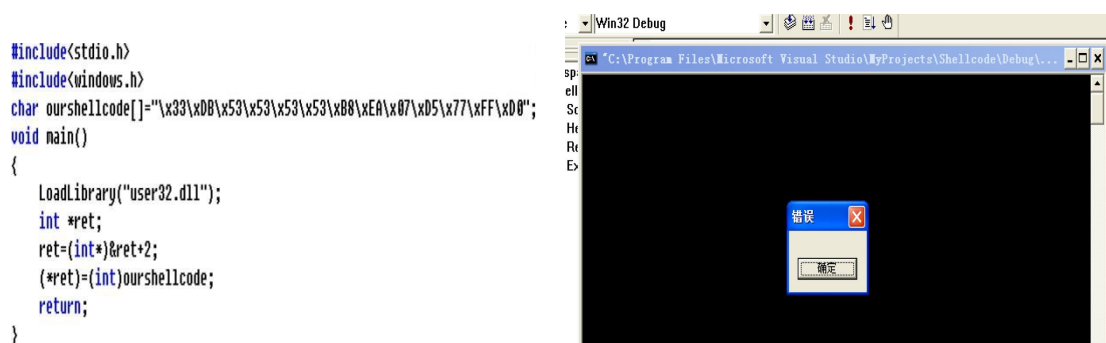
接着，设置断点调试进入反汇编，如右上图所示。可以看到 asm 代码部分的指令地址为 00401028—00401034，调出内存空间查看窗口，输入 00401028，我们可以查出该部分的机器指令，如下图所示。

|          |                                                 |
|----------|-------------------------------------------------|
| Address: | 0x00401028                                      |
| 00401028 | 33 DB 53 53 53 53 B8 EA 07 D5 77 FF D0 5F 5E 5B |
| 00401038 | 83 C4 40 3B EC E8 1E 00 00 00 8B E5 5D C3 CC CC |

借助记事本的全替换功能，可以得到对应的机器码：

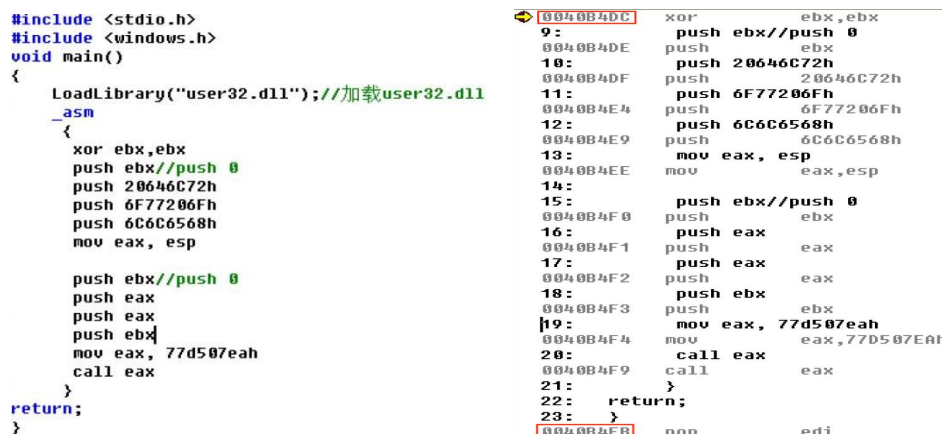
\x33\xDB\x53\x53\x53\x53\xB8\xEA\x07\xD5\x77\xFF\xD0

此时，我们完成了 Shellcode 代码的提取。可以用如左下图所示的代码来验证提取的 Shellcode 代码是否正确。运行，弹窗出现（如右下图所示），证明 shellcode 代码正确。



## 2. Shellcode 编码解码

对 shellcode 进行编码之后，才能进行攻击。在编码解码之前，需要先有一段 shellcode 代码。按照课上的讲解，可以得到“Hello World”的 shellcode 代码。如下图所示：



查询 0040B4DC—0040B4FA，再\x 替换空格，我们得到 shellcode 代码：

```
\x33\xDB\x53\x68\x72\x6C\x64\x20\x68\x6F\x20\x77\x6F\x68\x68\x65\x6C\x6C\x8B\xC4\x53\x50\x50\x53\xB8\xEA\x07\xD5\x77\xFF\xD0
```

接下来将其作为编码对象，用如下所示代码进行 xor 编码。大致过程为：取 0x44 为密钥 key，对 shellcode 代码每一位进行与 key 的异或操作，并保存编码后的 shellcode。

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

void encoder(char* input, unsigned char key)
{
    int i = 0, len = 0;

    FILE * fp; len = strlen(input);

    unsigned char * output = (unsigned char *)malloc(len + 1);

    for (i = 0; i < len; i++) output[i] = input[i] ^ key;

    fp = fopen("encode.txt", "w+"); fprintf(fp, "");

    for (i = 0; i < len; i++)
    {
        fprintf(fp, "\\x%.2x", output[i]);
        if ((i + 1) % 16 == 0) fprintf(fp, "\\n\\");
    }

    fprintf(fp, ""); fclose(fp);

    printf("dump the encoded shellcode to encode.txt OK!\\n");

    free(output);
}

int main()
{
    char sc[]
= "\\x33\\xDB\\x53\\x68\\x72\\x6C\\x64\\x20\\x68\\x6F\\x20\\x77\\x6F\\x68\\x68\\x65\\x6C\\x6C\\x8B\\xC4\\x53\\x50\\x50\\x53\\xB8\\xEA\\x07\\xD5\\x77\\xFF\\xD0";

    encoder(sc, 0x44);

    getchar();

    return 0;
}
```

```
int i = 0, len = 0;
FILE * fp;
len = strlen(input);
unsigned char * output = (uns...
for (i = 0; i < len; i++)
    output[i] = input[i] ^ key;
fp = fopen("encode.txt", "w+");
fprintf(fp, "");
for (i = 0; i < len; i++)
{
    fprintf(fp, "\\x%.2x", ou...
    if ((i + 1) % 16 == 0)
        fprintf(fp, "\\n\\");
}
fprintf(fp, "");
fclose(fp);
printf("dump the encoded shel...
free(output);
}
int main()
{
    char sc[] = "\\x33\\x0B\\x53\\x68\\...
    encoder(sc, 0x44);
    getchar();
    return 0;
}
```



在项目文件夹下可以找到生成的 encode.txt，打开可以看到编码后的 shellcode 代码：

```
\x77\x9f\x17\x2c\x36\x28\x20\x64\x2c\x2b\x64\x33\x2b\x2c\x2c\x21\x28\x28\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94
```

接下来还需要进行解码，才能最后实现攻击。所以我们先写出一个能对它后面的代码进行异或解码的 C 语言代码，把它提取成 shellcode 加到 Hello World 的 shellcode 前，就组合成

了完整的 shellcode 代码。解码代码如左下图所示：

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
int main()
{
    __asm
    {
        call table;
        table: pop eax;
        add eax, 0x15 ;越过decoder记录shellcode起始地址
        xor ecx, ecx
        decode_loop:
            mov bl, [eax + ecx]
            xor bl, 0x44 ;用0x44作为key
            mov [eax + ecx], bl
            inc ecx
            cmp bl, 0x90 ;末尾放一个0x90作为结束符
            jne decode_loop
    }
    return 0;
}
```

```
8:      call table;
00401038  call     table (0040103d)
9:      table: pop eax;
0040103D  pop     eax
10:     add eax, 0x15 ;越过decoder记录shellcode起始地址
0040103E  add     eax, 15h
11:     xor ecx, ecx
00401041  xor     ecx, ecx
12:     decode_loop:
13:     mov bl, [eax + ecx]
00401043  mov     bl, byte ptr [eax+ecx]
14:     xor bl, 0x44 ;用0x44作为key
00401046  xor     bl, 44h
15:     mov [eax + ecx], bl
00401049  mov     byte ptr [eax+ecx], bl
16:     inc ecx
0040104C  inc     ecx
17:     cmp bl, 0x90 ;末尾放一个0x90作为结束符
0040104D  cmp     bl, 90h
18:     jne decode_loop
00401050  jne     decode_loop (00401043)
19:     }
20:     return 0;
00401052  xor     eax, eax

Address: 0x00401038
00401038  E8 00 00 00 00 58 83 C0 15 33 C9 8A 1C 08 80 F3 44 88 1C 08
0040104C  41 80 FB 90 75 F1 33 C0 5F 5E 50 83 C4 40 30 EC E8 4F 24 00
00401060  00 8B E5 5D C3 CC CC CC CC CC CC CC CC CC CC CC CC CC CC
```

在这段代码中，call table 将 eip 的值（pop eax）压入栈中，接着 pop 操作就把 eax 的值赋值成了当前指令的地址，然后+ 0x15（解码部分代码长），也就定位到了紧跟其后需要解码的 shellcode 部分的起点；接着，decode\_loop 循环不断将代码异或 0x44 来解码，最后通过 0x90 以跳出循环。反汇编后如右上图所示，可以提取出这部分（解码程序）的 shellcode：

`\xE8\x00\x00\x00\x00\x58\x83\xC0\x15\x33\xC9\x8A\x1C\x08\x80\xF3\x44\x88\x1C\x08\x41\x80\xFB\x90\x75\xF1`

把它加到 Hello World 的 shellcode 前合并成一段代码，填进左下程序进行验证，发现弹窗输出“Hello World”，如右下图所示，验证正确。

```
#include <stdio.h>
#include <windows.h>
char ourshellcode[]=
"\xE8\x00\x00\x00\x00\x58\x83\xC0\x15\x33\xC9\x8A\x1C\x08\x80\xF3\x44\x88\x1C\x08\x41\x80\xFB\x90\x75\xF1\x77\x9f\x17\x2c\x36\x28\x20\x64\x2c\x2b\x64\x33\x2b\x2c\x2c\x21\x28\x28\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94\xd4";
void main() {
    LoadLibrary("user32.dll");
    int *ret;
    ret=(int*)&ret+2;
    (*ret)=(int)ourshellcode;
    return; }
```

`:\xE8\x00\x58\x83\xC0\x15\x33\xC9\x8`



## 心得体会：

- 通过本次实验，更具体、更深入地了解了 shellcode 代码的提取、编写、编码以及解码。用 shellcode 精心设计，利用溢出覆盖返回地址，就可以使程序去执行植入的恶意代码。
- 学习了一种编写 shellcode 代码的简便方式：在 C 语言程序中插入汇编语言，反汇编查看对应汇编语言的地址，再查找地址中的机器码，就实现了 shellcode 的提取编写。
- Xor 编码就是 shellcode 代码逐位与密钥异或，对应的解码，就是能将后面代码逐位与密钥 xor 的程序的 shellcode 与待解码段的拼接。
- 了解到，可以巧用 call table 获取当前代码的地址。